



**KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY
(AN AUTONOMOUS INSTITUTION)**

**Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH,
Narayanaguda, Hyderabad-500029.**



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

LAB MANUAL

NAME OF THE LAB: SKILL DEVELOPMENT COURSE-III

**B.Tech IV YEAR I SEM (KR23)
ACADEMIC YEAR 2026-27**



KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY
(AN AUTONOMOUS INSTITUTION)

**Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH,
Narayanguda, Hyderabad – 500029**



Certificate

This is to certify that the following is a Bonafide Record of the workbook task done by

_____ bearing Roll No _____ of

_____ Branch of _____ year

B.Tech Course in the _____ Subject

during the Academic year _____ & _____ under our supervision.

Number of experiments completed: _____.

Signature of Staff Member Incharge

Signature of Head of the Dept.

Signature of Internal Examiner

Signature of External Examiner

INDEX

S.NO	CONTENTS	PAGE NO
I	Vision/Mission/ POs/ PSOs/ PEOs	
II	Syllabus	
III	Course Outcomes, CO-PO Mapping.	
EXP NO.	EXPERIMENTS	PAGE NO
-	Creating a Simple React Application in VS Code.	
1	Build a React application to demonstrate JSX and Virtual DOM.	
2	Create a product listing page using multiple stateless React components. The page should include a header, product cards displaying name, price, and category, and a “Buy Now” button that responds to user interactions using props.	
3	Develop an inventory management application using React Hooks such as useState, useEffect, useContext, useRef, and useMemo. The application should allow adding products, searching products, calculating total inventory value, and toggling between light and dark themes dynamically.	
4	Build a simplified e-commerce application using React Router. The application should include Home, Products, Product Details, Contact, Login, Cart, and 404 pages with dynamic routing, protected routes, and navigation links.	
5	Create a React form where input fields are dynamically generated from a JavaScript configuration object. The form should store user input using state and display the submitted data below the form.	
6	Develop a React single-page application with routing between Home and Users pages. Fetch user data from a public API using useEffect and useState, display the data conditionally, and show a loading indicator while data is being fetched.	



KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY (AN AUTONOMOUS INSTITUTION)

Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH,
Narayanguda, Hyderabad – 500029



Department of Computer Science and Engineering

Vision of the Institution:

To be the fountain head of latest technologies, producing highly skilled, globally competent engineers.

Mission of the Institution:

- To provide a learning environment that inculcates problem solving skills, professional, ethical responsibilities, lifelong learning through multi modal platforms and prepare students to become successful professionals.
- To establish Industry Institute Interaction to make students ready for the industry.
- To provide exposure to students on latest hardware and software tools.
- To promote research based projects/activities in the emerging areas of technology convergence.
- To encourage and enable students to not merely seek jobs from the industry but also to create new enterprises
- To induce a spirit of nationalism which will enable the student to develop, understand India's challenges and to encourage them to develop effective solutions.
- To support the faculty to accelerate their learning curve to deliver excellent service to students.



KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY
(AN AUTONOMOUS INSTITUTION)

Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH,
Narayanguda, Hyderabad – 500029



Department of Computer Science and Engineering

Vision of the Department:

- To be among the region's premier teaching and research Computer Science and Engineering departments producing globally competent and socially responsible graduates in the most conducive academic environment.

Mission of the Department:

- To provide faculty with state-of-the-art facilities for continuous professional development and research, both in foundational aspects and of relevance to emerging computing trends.
- To impart skills that transform students to develop technical solutions for societal needs and inculcate entrepreneurial talents.
- To inculcate an ability in students to pursue the advancement of knowledge in various specializations of Computer Science and Engineering and make them industry-ready.
- To engage in collaborative research with academia and industry and generate adequate resources for research activities for seamless transfer of knowledge resulting in sponsored projects and consultancy.
- To cultivate responsibility through sharing of knowledge and innovative computing solutions that benefits the society-at-large.
- To collaborate with academia, industry and community to set high standards in academic excellence and in fulfilling societal responsibilities.



KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY
(AN AUTONOMOUS INSTITUTION)

**Accredited by NBA & NAAC, Approved by AICTE, Affiliated to
JNTUH, Narayanguda, Hyderabad – 500029**



Department of Computer Science and Engineering

PROGRAM OUTCOMES (POs):

PO1: Engineering Knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3: Design/Development of Solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4: Conduct Investigations of Complex Problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5: Modern Tool Usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6: The Engineer and Society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7: Environment and Sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8: Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9: Individual and Team Work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10: Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11: Project Management and Finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12: Life-long Learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.



KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY **(AN AUTONOMOUS INSTITUTION)**

**Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH,
Narayanguda, Hyderabad – 500029**



Department of Computer Science and Engineering

PROGRAM SPECIFIC OUTCOMES (PSOs)

PSO1: An ability to analyze the common business functions to design and develop appropriate Computer Science solutions for social upliftments.

PSO2: Shall have expertise on the evolving technologies like Python, Machine Learning, Deep Learning, Internet of Things (IOT), Data Science, Full stack development, Social Networks, Cyber Security, Big Data, Mobile Apps, CRM, ERP etc.



KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY
(AN AUTONOMOUS INSTITUTION)

Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH,
Narayanguda, Hyderabad – 500029



Department of Computer Science and Engineering

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

PEO1: Graduates will have successful careers in computer related engineering fields or will be able to successfully pursue advanced higher education degrees.

PEO2: Graduates will try and provide solutions to challenging problems in their profession by applying computer engineering principles.

PEO3: Graduates will engage in life-long learning and professional development by rapidly adapting changing work environment.

PEO4: Graduates will communicate effectively, work collaboratively and exhibit high levels of professionalism and ethical responsibility.



KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY
(AN AUTONOMOUS INSTITUTE)

Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH,
Narayanguda, Hyderabad – 500029



Department of Computer Science & Engineering
B.Tech. IV Year I Semester Syllabus (KR23)

SKILL DEVELOPMENT COURSE -III (23CS705PC)
Common to CSE, CSE (DS), CSE (AI&ML), IT

L T P C
0 0 2 1

Prerequisite/ Co-requisites:

1. 23CS503PC - Web Technologies Course

Course Objectives: The course will help to

1. Provide an understanding of the fundamentals of ReactJS for developing modern web applications.
2. Explain the concepts of JSX, components, props, and state for building reusable user interface components.
3. Enable students to use React Hooks for effective state management and handling side effects.
4. Introduce client-side routing techniques using React Router for single-page applications.
5. Develop the ability to build interactive, data-driven applications using forms, conditional rendering, and API integration.

Course Outcomes: After learning the concepts of the course, the student is able to

1. Understand the core concepts of ReactJS and develop applications using JSX and functional components.
2. Apply props and state to manage data flow and user interactions in React applications.
3. Use React Hooks such as useState and useEffect to manage component state and lifecycle behavior.
4. Implement routing and navigation in single-page applications using React Router.
5. Design and develop complete ReactJS applications that consume external APIs and handle user input effectively.

List of Experiments:

1. Build a React application to demonstrate JSX and Virtual DOM.
2. Create a product listing page using multiple stateless React components. The page should include a header, product cards displaying name, price, and category, and a “Buy Now” button that responds to user interactions using props.
3. Develop an inventory management application using React Hooks such as useState, useEffect,

useContext, useRef, and useMemo. The application should allow adding products, searching products, calculating total inventory value, and toggling between light and dark themes dynamically.

4. Build a simplified e-commerce application using React Router. The application should include Home, Products, Product Details, Contact, Login, Cart, and 404 pages with dynamic routing, protected routes, and navigation links.
5. Create a React form where input fields are dynamically generated from a JavaScript configuration object. The form should store user input using state and display the submitted data below the form.
6. Develop a React single-page application with routing between Home and Users pages. Fetch user data from a public API using useEffect and useState, display the data conditionally, and show a loading indicator while data is being fetched.

TEXTBOOKS:

1. Learning React, alex banks, Eve Porcello, O'Reilly Media.
2. React Up & Running, Stoyan Stefanov, O'Reilly Media.

REFERENCE BOOKS:

- Full stack React, Anthony Accomazzo et al., Fullstack.io.
- Pro React, Adam Freeman, Apress.
- The Road to Learn React, Robin Wieruch.



Department of Computer Science & Engineering

Course Outcomes and CO-PO-PSO Mapping

Course Outcomes:

Course Outcome (CO)	Comments
CO1	Understand the core concepts of ReactJS and develop applications using JSX and functional components.
CO2	Apply props and state to manage data flow and user interactions in React applications.
CO3	Use React Hooks such as useState and useEffect to manage component state and lifecycle behavior.
CO4	Implement routing and navigation in single-page applications using React Router.
CO5	Design and develop complete ReactJS applications that consume external APIs and handle user input effectively.

CO-PO-PSO MAPPING:

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2
CO1	3	2	2	–	3	–	–	–	–	1	–	1	2	3
CO2	3	3	3	–	3	–	–	–	1	1	–	1	2	3
CO3	3	3	3	1	3	–	–	–	1	1	–	2	2	3
CO4	3	2	3	–	3	–	–	–	1	1	–	1	2	3
CO5	3	3	3	2	3	–	–	–	2	2	1	2	3	3

Creating a Simple React Application in VS Code (Step-by-Step)

This guide explains how to create your first React application using Visual Studio Code (VS Code).

Softwares Required:- Visual Studio Code (VS Code), Node.js, React.js, Browser.

Prerequisites: Before creating a React application, install the following software.

1. Install Node.js

React requires Node.js because it includes npm (Node Package Manager).

Visit: <https://nodejs.org>

Download the LTS (Long-Term Support) version.

Install using the default settings.

Verify Installation

Open Command Prompt or Terminal. Use command

`node -v`

Example Output: v22.5.1

To Check npm

`npm -v`

Example:- 10.8.2

2. Install Visual Studio Code

Download and install VS Code from

<https://code.visualstudio.com/>

3. Install VS Code Extensions

Recommended extensions:

ES7+ React/Redux Snippets

Prettier

ESLint

Auto Rename Tag

Auto Close Tag

Step 1: Open VS Code

Launch Visual Studio Code.

Step 2: Open Terminal

Go to

Terminal → New Terminal or press Ctrl + `

A terminal appears at the bottom.

Step 3: Navigate to Your Project Folder

Example:- cd Desktop or cd Documents

Step 4: Create a React Project

There are two common methods.

Method 1 (Recommended): Using Vite

Run

```
npm create vite@latest my-react-app
```

You'll see:

Project name:

Press Enter.

Then select

React

Next select

JavaScript

Example:-

Project Name : my-react-app

Framework: React

Variant: JavaScript

Method 2 (Older Method)

```
npx create-react-app my-react-app
```

This method works but is slower.

Today, Vite is recommended.

Step 5: Move Into the Project Folder

```
cd my-react-app
```

Step 6: Install Dependencies

npm install

or simply

npm i

Wait until installation finishes.

Step 7: Open Project in VS Code

If you're already inside the folder:

`code .`

The project opens in VS Code.

If the command doesn't work, open VS Code manually and choose

File → Open Folder

Select

my-react-app

Step 8: Start the Development Server

Run

`npm run dev`

Output: VITE v7.x.x

Local:

`http://localhost:5173/`

Open `http://localhost:5173`

The default React page appears.

React Project Structure

my-react-app

```
|
|
|— node_modules
|— public
|— src
|   |— assets
|   |— App.css
|   |— App.jsx
|   |— index.css
|   |— main.jsx
|
|— package.json
|— vite.config.js
```

└─ index.html

Important Files

src/ → Contains all React source code.

App.jsx

Main component.

Example:

```
function App() {  
  return (  
    <h1>Hello React!</h1>  
  );  
}  
export default App;
```

main.jsx → Application entry point.

```
import React from "react";  
import ReactDOM from "react-dom/client";  
import App from "./App";
```

```
ReactDOM.createRoot(document.getElementById("root")).render(  
  <App />  
);
```

package.json Contains:

dependencies

project name

scripts

version

Example

```
{  
  "name": "my-react-app",  
  "scripts": {  
    "dev": "vite"  
  }  
}
```

```
}
```

Step 9: Edit the Application

Open src/**App.jsx**

Replace the code with

```
function App() {  
  return (  
    <div>  
      <h1>Welcome to React</h1>  
      <p>This is my first React application.</p>  
    </div>  
  );  
}  
export default App;
```

Save the file.

The browser updates automatically due to Hot Module Replacement (HMR).

Step 10: Run Again (If Necessary)

Whenever you reopen the project:

Open Terminal

cd my-react-app

Start **npm run dev**

Open <http://localhost:5173>

Creating Your First Component

Inside src/components

Create **Welcome.jsx**

```
function Welcome() {  
  return (  
    <h2>Welcome Component</h2>  
  );  
}  
export default Welcome;
```

→ Import it into App.jsx

```
import Welcome from "../components/Welcome";
```

```
function App() {
  return (
    <div>
      <h1>React Application</h1>
      <Welcome />
    </div>
  );
}
export default App;
```

Output:-

React Application
Welcome Component

Useful npm Commands

Command	Purpose
npm install	Install project dependencies
npm run dev	Start the development server
npm run build	Build the production version of the application
npm run preview	Preview the production build locally
npm update	Update installed packages to newer compatible versions
npm -v	Check the installed npm version
node -v	Check the installed Node.js version

Common Errors and Solutions

1. 'node' is not recognized

Cause: Node.js is not installed or not added to the system PATH.

Solution: Reinstall Node.js and ensure the "Add to PATH" option is selected during installation.

2. 'npm' is not recognized

Cause: npm is unavailable because Node.js installation is incomplete.

Solution: Reinstall Node.js and verify the installation using node -v and npm -v.

3. 'code' is not recognized

Cause: VS Code's command-line tool is not added to the PATH.

Solution: In VS Code, press Ctrl + Shift + P, type Shell Command: Install 'code' command in PATH, execute it, then restart the terminal.

4. Port 5173 is already in use

Cause: Another application is using the default Vite port.

Solution: Stop the conflicting process or start Vite on a different port:

```
npm run dev -- --port 3000
```

5. Module not found

Cause: Incorrect file path or missing file.

Solution: Verify the file exists and check the import statement for the correct relative path.

Summary:-

Install Node.js and VS Code.

Open VS Code and launch a terminal.

Create a React project using Vite.

Navigate to the project folder and install dependencies.

Start the development server with `npm run dev`.

Open the local URL (typically `http://localhost:5173`).

Edit `App.jsx` to build your application.

Create reusable components inside the `src/components` folder.

Save your files and see changes instantly with Hot Module Replacement (HMR).

EXERCISE-1

Build a simple React application from scratch that demonstrates both JSX and the Virtual DOM.

Aim:- To build a simple React application that demonstrates JSX for creating UI elements and the Virtual DOM for efficient updating of the user interface using the `useState` hook.

Explanation:-

1. JSX (JavaScript XML) allows writing HTML-like code inside JavaScript, making React components easy to read and write.
2. Virtual DOM is a lightweight copy of the Real DOM. When the state changes using `useState`, React updates only the modified part of the page instead of reloading the entire page, improving performance.

Algorithm / Steps Followed

1. Create a React application.
2. Import the `useState` hook from React.
3. Initialize a state variable `count` with value `0`.
4. Display a heading and the current counter value using JSX.
5. Add a button with an `onClick` event.
6. Increment the counter using `setCount(count + 1)` when the button is clicked.
7. React updates only the changed counter value using the Virtual DOM.

Expected Output

The page displays the heading "React JSX & Virtual DOM Demo".

Initially, the counter value is 0.

Each click on the Increment button increases the counter by 1 without reloading the page.

Step 1: Create a React Project

```
npm create vite@latest jsx-virtual-dom-demo
```

→ Here Project Name is **jsx-virtual-dom-demo**

Choose:

- **Framework:** React
- **Variant:** JavaScript
- **Which linter to use?:** ESLint

Move into the project and install dependencies

In case if development server doesn't start automatically. Then run command:

npm run dev

Open the URL shown in the terminal (usually <http://localhost:5173>).

If you are using for the 2nd time, then to **Move into the Project** use command

cd jsx-virtual-dom-demo

To Start the Development Server use command:-

npm run dev

You'll see something like: Local: <http://localhost:5173/>

Open it in your browser. You should see the default React + Vite page.

Step 2: Project Structure

JSX-VIRTUAL-DOM-DEMO

```
├── public
├── src
│   ├── App.css
│   ├── App.jsx
│   ├── index.css
│   └── main.jsx
├── package.json
├── index.html
└── vite.config.js
```

→ Open **App.jsx**

Replace the code with

```
import { useState } from "react";
```

```
function App() {
  // useState is used when data changes and the UI needs to update.
  const [count, setCount] = useState(0);
```

```
  return (
    <div>
      {/* JSX: HTML-like syntax inside JavaScript */}
      <h1>React JSX & Virtual DOM Demo</h1>

      {/* JSX Expression */}
      <h2>Counter: {count}</h2>
```

```
    {/* Clicking the button updates the state.
       React updates only the changed part (Counter)
    */}
  )
}
```

```

        using the Virtual DOM instead of reloading the page. */}
        <button onClick={() => setCount(count + 1)}>
          Increment
        </button>
      </div>
    );
  }
}
export default App;

```

Initial Output:-



React JSX & Virtual DOM Demo

Counter: 0

Increment

After Clicking on Increment Button Counter value updates automatically



React JSX & Virtual DOM Demo

Counter: 1

Increment

[OR]

Open App.jsx

replace the code with

```

import './App.css';

import { useState } from 'react';

function App() {

  // -----

  // JSX Variables Demo

  // -----

```

```
const name = "John";

const course = "React";

const year = 2026;


// -----

// Conditional Rendering Demo

// Change to false to display "Please Login"

// -----

const isLoggedIn = true;


// -----

// Array Demo

// -----

const fruits = ["Apple", "Banana", "Mango", "Orange"];


// Find the position of Mango

const mangoIndex = fruits.findIndex((fruit) => fruit === "Mango");


// -----

// useState Hook

// -----

// useState is used when a value needs to change dynamically

// and the UI should update automatically.

// Syntax:

// const [stateVariable, setStateVariable] = useState(initialValue);
```

```
// Example:

// count -> current state value

// setCount -> function used to update count

// 0 -> initial value of count

const [count, setCount] = useState(0);

return (

  <div style={{ margin: "20px", fontFamily: "Arial" }}>
```

```
    {/* ----- */}
```

```
    {/* 1. Basic JSX */}
```

```
    {/* ----- */}
```

```
    <h1>React JSX Demo</h1>
```

```
    <p>Welcome to my React application.</p>
```

```
    <hr />
```

```
    {/* ----- */}
```

```
    {/* 2. JSX Variables & Expressions */}
```

```
    {/* ----- */}
```

```
    <h2>JSX Demonstration</h2>
```

```
    <h3>Hello, {name}!</h3>
```

```
    <p>Welcome to the {course} course.</p>
```

```
    <p>Current Year: {year}</p>
```

```
    <p>2 + 3 = {2 + 3}</p>
```

```
    <hr />
```

```
{/* ----- */}
```

```
{/* 3. Conditional Rendering */}
```

```
{/* ----- */}
```

```
<h2>Conditional Rendering</h2>
```

```
{isLoggedIn ? (
```

```
  <h3>Welcome Back!</h3>
```

```
): (
```

```
  <h3>Please Login</h3>
```

```
)}
```

```
<hr />
```

```
{/* ----- */}
```

```
{/* 4. Rendering List using map() */}
```

```
{/* ----- */}
```

```
<h2>Fruit List (Unordered List)</h2>
```

```
<ul>
```

```
  {fruits.map((fruit, index) => (
```

```
    <li key={index}>{fruit}</li>
```

```
  )})
```

```
</ul>
```

```
<hr />
```

```
{/* ----- */}
```

```
{/* 5. Ordered List with Index */}
```

```
{/* ----- */}
```

```
<h2>Fruit List with Index</h2>
```

```
<ol>
```

```
{fruits.map((fruit, index) => (
```

```
  <li key={index}>
```

```
    {index}: {fruit}
```

```
  </li>
```

```
))}
```

```
</ol>
```

```
<hr />
```

```
{/* ----- */}
```

```
{/* 6. Find Index Example */}
```

```
{/* ----- */}
```

```
<h2>Fruit Position</h2>
```

```
<p>Mango is at index: {mangoIndex}</p>
```

```
<hr />
```

```
{/* ----- */}
```

```
{/* 7. Virtual DOM Demo using useState */}
```

```
{/* ----- */}
```

```
<h2>Virtual DOM Demo</h2>
```

```
<h3>Counter: {count}</h3>
```

```
<button onClick={() => setCount(count + 1)}>
```

```
  Increment
```

```
</button>
```

```
{/*
```

When the button is clicked:

1. `setCount(count + 1)` updates the state.
2. React creates a new Virtual DOM.
3. React compares it with the previous Virtual DOM (Diffing).
4. Only the changed part (Counter value) is updated in the Real DOM.
5. The entire page is NOT reloaded, making React fast and efficient.

```
*/}
```

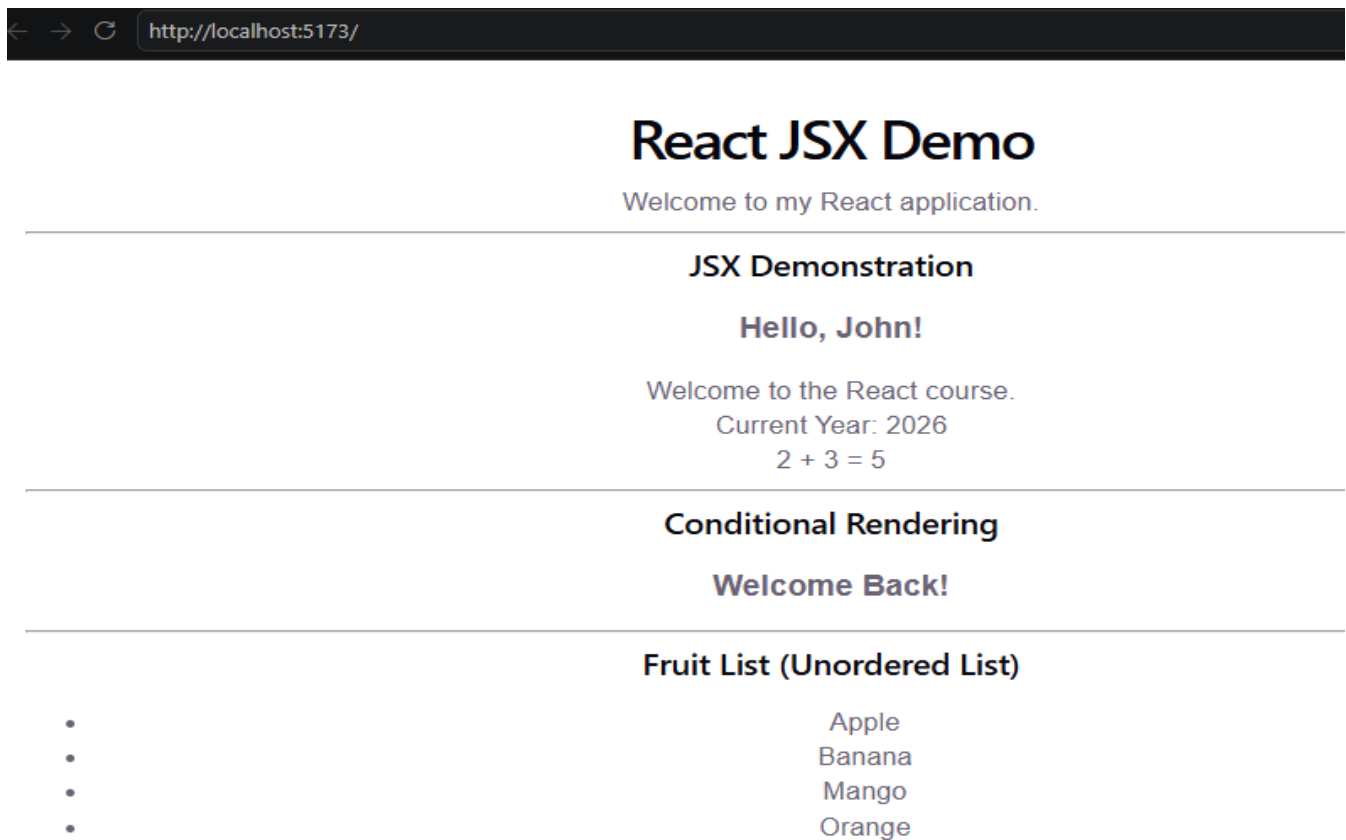
```
</div>
```

```
);
```

```
}
```

```
export default App;
```

Output:-



Fruit List with Index

1.	0: Apple
2.	1: Banana
3.	2: Mango
4.	3: Orange

Fruit Position

Mango is at index: 2

Virtual DOM Demo

Counter: 1

Increment

Try using:- `<React.Fragment>` instead of `<div>` in return function.

Example:-

```
import React from "react";
function App() {
  return (
    <React.Fragment>
      <h1>Hello</h1>
      <p>Welcome</p>
    </React.Fragment>
  );
}
```

→ A fragment acts like an invisible parent. It satisfies React's requirement for a single root element without adding an extra HTML element to the page.

Summary:-

1. Create an Array

An array (fruits) is used to store multiple values.

```
const fruits = ["Apple", "Banana", "Mango", "Orange"];
```

- Creates an array named fruits.
- The array contains four fruit names.
- Each item in the array can be displayed on the webpage.

2. Display Array Items Using map()

The map() method is used to iterate through the array and display each item.

```
{ fruits.map((fruit, index) => (  
  <li key={index}>  
    {index}: {fruit}  
  </li>  
))}
```

- map() loops through each item in the array.
- fruit represents the current fruit name.
- index represents the position of the current item.
- creates a list item.
- {fruit} displays the fruit name.
- {index} displays the position of the fruit.

Using the key Attribute

```
<li key={index}>{index}: {fruit}</li>
```

- The key prop gives each list item a unique identifier.
- React uses the key prop to efficiently update and re-render list items.
- This example demonstrates how the map() method is used to render lists in JSX.

3. Display the Index Position of "Mango"

Use the findIndex() method to find the position of "Mango" in the array.

```
const mangoIndex = fruits.findIndex((fruit) => fruit === "Mango");
```

```
<p>Mango is at index: {mangoIndex}</p>
```

- findIndex() searches the array for the specified element.
- It returns the index of "Mango" if found.
- If the element is not found, it returns -1.

→For **incrementing the counter value** use code:

```
<button onClick={() => setCount(count + 1)}>
```

➕ Increment

```
</button>
```

→To **decrement the counter value** use code:

```
<button onClick={() => setCount(count - 1)}>
```

— Decrement

</button>

→ To **reset the counter value** use code:

<button onClick={() => setCount(0)}>

 Reset

</button>

EXERCISE-2

Create a product listing page using multiple stateless React components. The page should include a header, product cards displaying name, price, and category, and a "Buy Now" button that responds to user interactions using props.

Aim:- To create a React product listing page using multiple stateless components that displays product details and handles button click events using props.

Explanation:-

1. Header component displays the page title.
2. ProductCard component displays the product name, price, category, and a Buy Now button.
3. App component passes product data and the `buyProduct()` function to `ProductCard` using props.
4. Clicking Buy Now displays the selected product name using an alert.

Components Used:-

1. `Header.jsx` – Displays the heading.
2. `ProductCard.jsx` – Displays product information.
3. `App.jsx` – Combines components and passes props.
4. `main.jsx` – Renders the React application.

→ This is a common React lab exercise that demonstrates:

- Stateless (Functional) Components
- Props
- Component Reusability
- Event Handling
- Parent-Child Communication

Expected Output:-

A product listing page with multiple product cards, where clicking the Buy Now button shows the selected product name.

Step 1: Create a React Project

npm create vite@latest product-listing

Choose:

- **Framework:** React
- **Variant:** JavaScript
- **Which linter to use?:** ESLint

Move into the project and install dependencies

Then run command: **npm run dev**

in case if development server doesn't start automatically.

Then open the URL shown in the terminal (usually <http://localhost:5173>).

Step 2: Create the Project Structure

Inside the src folder, create these files:

```
src/
├── App.jsx
├── Header.jsx
├── ProductCard.jsx
├── App.css
└── main.jsx
```

Step 3: Create the Header Component

Create a file named **Header.jsx**.

```
// Header component
function Header() {
  return (
    <header style={styles.header}>
      <h1>My Product Store</h1>
    </header>
  );
}
```

```
// Inline CSS styles
const styles = {
  header: {
    backgroundColor: "#4CAF50",
    color: "white",
    padding: "20px",
    textAlign: "center",
  },
};
```

```
export default Header;
```

Step 4: Create the Product Card Component

Create another file **ProductCard.jsx**

```
// ProductCard receives data through props

function ProductCard(props) {

  return (
    <div style={styles.card}>
      <h2>{props.name}</h2>

      <p>
        <strong>Price:</strong> ₹ {props.price}
      </p>

      <p>
        <strong>Category:</strong> {props.category}
      </p>

      { /* Calls the buyProduct function from App.jsx */ }
      <button onClick={() => props.buyProduct(props.name)}>
        Buy Now
      </button>
    </div>
  );
}

// Inline styles for the product card
const styles = {
  card: {
    border: "1px solid gray",
    borderRadius: "10px",
    padding: "20px",
    width: "250px",
    textAlign: "center",
    margin: "15px",
    boxShadow: "2px 2px 8px gray",
  },
};
export default ProductCard;
```

Step 5: Create the Parent Component

→ Open App.jsx and replace code

```

import Header from "./Header";
import ProductCard from "./ProductCard";
import "./App.css";

function App() {
  // Function passed to ProductCard as a prop
  const buyProduct = (productName) => {
    alert(`You selected: ${productName}`);
  };

  return (
    <div>
      {/* Displays the page header */}

      <Header />

      {/* Container for all product cards */}

      <div className="products">

        <ProductCard
          name="Laptop"
          price="55000"
          category="Electronics"
          buyProduct={buyProduct}/>

        <ProductCard
          name="Smartphone"
          price="25000"
          category="Electronics"
          buyProduct={buyProduct}/>

        <ProductCard
          name="Shoes"
          price="3000"
          category="Fashion"
          buyProduct={buyProduct}/>

        <ProductCard
          name="Watch"
          price="5000"
          category="Accessories"
          buyProduct={buyProduct}/>
      </div>
    </div>
  );
}

export default App;

```

Step 6: Add CSS

Create **App.css**

```
/* Styles for the entire page */

body{

  margin:0;
  font-family:Arial, Helvetica, sans-serif;
  background:#f2f2f2;
}

/* Displays product cards in a responsive flex layout */
.products{
  display:flex;
  justify-content:center;
  flex-wrap:wrap;
  margin-top:30px;
}

/* Button styling */
button{
  background:#4CAF50;
  color:white;
  padding:10px 20px;
  border:none;
  cursor:pointer;
  border-radius:5px;
}

/* Button hover effect */
button:hover{
  background:#2e7d32;
}
```

Step 7: Create main.jsx

```
import { StrictMode } from "react";

import { createRoot } from "react-dom/client";

import App from "./App.jsx";

import "./App.css";

import "./index.css"; // Imports global CSS
```

```
// Renders the App component inside the root element

createRoot(document.getElementById('root')).render(

  <StrictMode>

    <App />

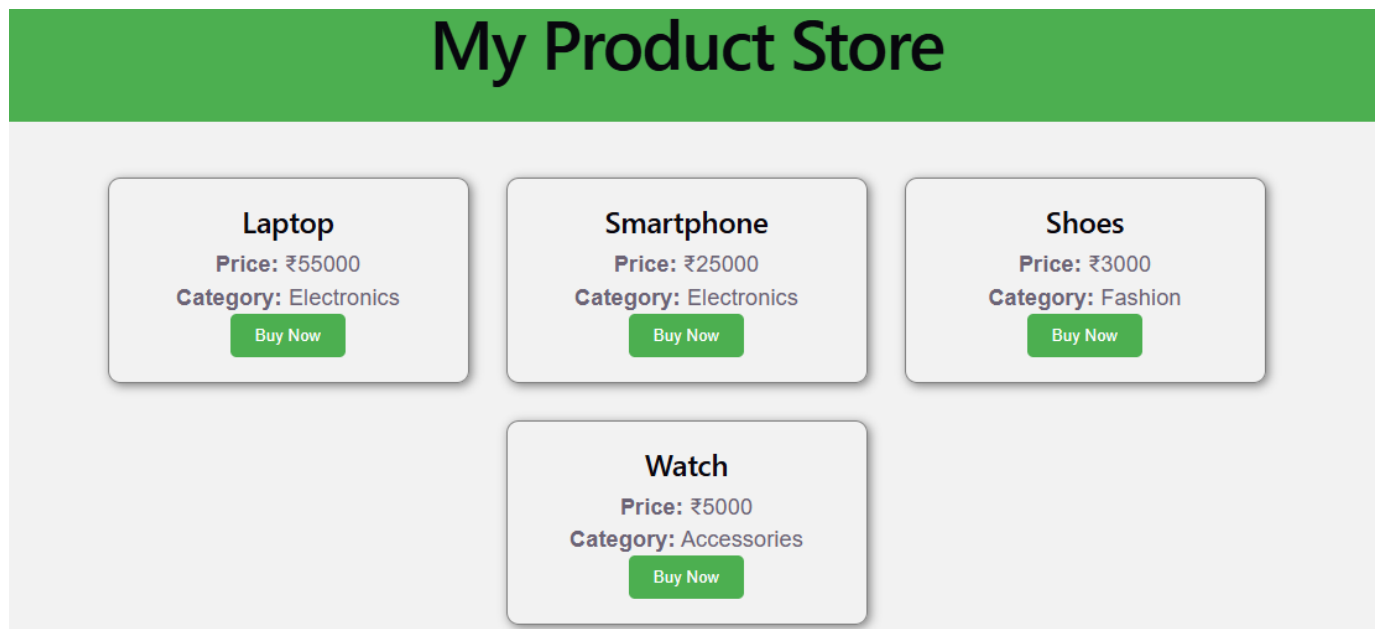
  </StrictMode>,

)
```

Step 8: Refresh the Browser or Run the Project using command:

npm run dev

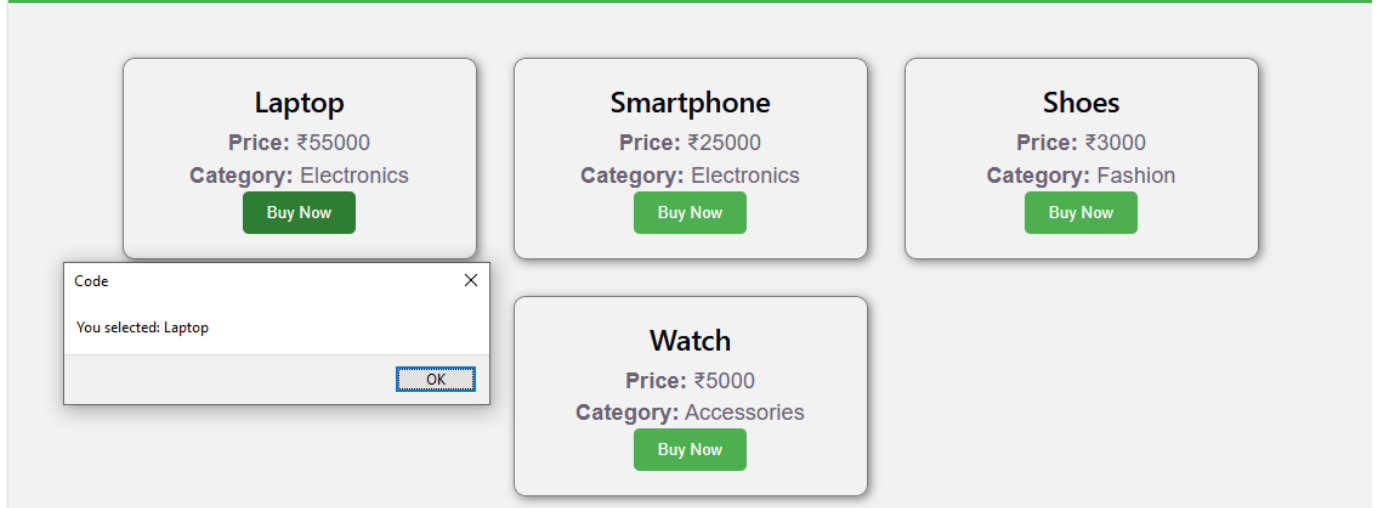
Output:



→ After clicking on Buy Now Button available in ProductCard Laptop, it displays alert message as shown below.

You selected: Laptop

My Product Store



Summary:- The **Buy Now** button:

```
<button onClick={() => props.buyProduct(props.name)}>  
  Buy Now  
</button>
```

When clicked,

- it calls the function `buyProduct`
- sends the product name to the parent component.

EXERCISE-3

Develop an inventory management application using React Hooks such as `useState`, `useEffect`, `useContext`, `useRef`, and `useMemo`. The application should allow adding products, searching products, calculating total inventory value, and toggling between light and dark themes dynamically.

→ **Inventory Management Application Using React Hooks**

Q:- What is inventory?

Inventory means **the products available in a shop or store.**

Example:

Product	Price	Quantity
Laptop	₹50,000	2
Mouse	₹500	5
Keyboard	₹800	3

Our application will allow us to:

- Add new products
- Search products
- Display all products
- Calculate the total inventory value
- Change between Light Mode and Dark Mode.

→ **Hooks** are special functions that let us use features like state and lifecycle in functional components. In simple words Hooks add extra functionality to functional components. Hooks make the code shorter, easier to understand, and allow us to use state without writing class components.

→ Five React Hooks that we use in our application are:-

- `useState`
- `useEffect`
- `useContext`
- `useRef`
- `useMemo`

→ **This project teaches how different React Hooks solve different problems:**

- **`useState`** stores changing data like products and form inputs.
- **`useEffect`** performs actions after data changes.
- **`useContext`** shares common data (the theme) across components without passing props through every

level.

- **useRef** gives direct access to a specific HTML element, such as focusing the search input.
- **useMemo** remembers expensive calculations, making searching and total value calculation more efficient.

Together, these hooks help build a responsive, organized, and efficient React application.

Step 1: Create a React Project

```
npm create vite@latest inventory-app
```

Choose:

- **Framework:** React
- **Variant:** JavaScript
- **Which linter to use?:** ESLint

Move into the project and install dependencies

In case if development server doesn't start automatically.

Then run command: **npm run dev**

Open the URL shown in the terminal (usually <http://localhost:5173>).

Step 2: Create the Project Structure

Inside the src folder, create these files:

Project Structure

```
inventory-app/  
├── src/  
│   ├── Components/  
│   │   ├── ProductForm.jsx  
│   │   ├── ProductList.jsx  
│   │   └── SearchBar.jsx  
│   ├── Context/  
│   │   └── ThemeContext.jsx  
│   ├── App.jsx  
│   └── main.jsx  
├── package.json  
└── vite.config.js
```

Step 3: Create Theme Context (useContext)

Purpose of ThemeContext.jsx:-

The ThemeContext file is used to:

- Store the application's theme (Light Mode or Dark Mode).
- Make the theme available to all components.
- Allow any component to change the theme without passing props.

→ Create file ThemeContext.jsx in Context folder (in src)

```
import { createContext, useState } from "react";

export const ThemeContext = createContext();

export function ThemeProvider({ children }) {

  const [darkMode, setDarkMode] = useState(false);

  const toggleTheme = () => {

    setDarkMode(prev => !prev);

  };

  return (

    <ThemeContext.Provider value={{ darkMode, toggleTheme }}>

      {children}

    </ThemeContext.Provider>

  );

}
```

Step 4: Add Products (useState)

→ Purpose of ProductForm.jsx

The **ProductForm** component is used to:

- Accept product details from the user.
- Store those details temporarily.
- Send the product to the parent component (App.jsx).

- Clear the form after adding the product.

ProductForm.jsx → Create file ProductForm.jsx in Components folder (in src)

```
import { useState } from "react";

function ProductForm({addProduct}) {

  const [name,setName] = useState("");

  const [price,setPrice] = useState("");

  const [quantity,setQuantity] = useState("");

  function submitHandler(e){

    e.preventDefault();

    addProduct({

      id:Date.now(),

      name,

      price:Number(price),

      quantity:Number(quantity)

    });

    setName("");

    setPrice("");

    setQuantity("");

  }

  return (

    <form onSubmit={submitHandler}>

      <h2>Add Product</h2>

      <input placeholder="Product Name"

        value={name}
```

```

        onChange={(e)=>setName(e.target.value)}

    />

    <input placeholder="Price"

        value={price}

        onChange={(e)=>setPrice(e.target.value)}

    />

    <input placeholder="Quantity"

        value={quantity}

        onChange={(e)=>setQuantity(e.target.value)}

    />

    <button>Add Product</button>

</form>

)

}

export default ProductForm;

```

Step 5: Search Products (useRef)

→ Purpose of SearchBar.jsx

The **SearchBar** component allows the user to:

- Type the name of a product to search.
- Send the search text to the parent component (`App.jsx`).
- Focus the search input when the **Search** button is clicked.

SearchBar.jsx --> Create file SearchBar.jsx in Components folder (in src)

```

import { useRef } from "react";

function SearchBar({ setSearch }) {

    const inputRef = useRef();

```

```

function focusInput(){
  inputRef.current.focus();
}

return (
  <div>
    <input ref={inputRef}
      placeholder="Search Product"
      onChange={(e)=>setSearch(e.target.value)}
    />
    <button onClick={focusInput}>Search</button>
  </div>
)
}

export default SearchBar;

```

Step 6: Display Products

→ Purpose of ProductList.jsx

The **ProductList** component is responsible for:

- Receiving the list of products from `App.jsx`.
- Displaying each product on the screen.
- Showing the product's **Name**, **Price**, and **Quantity**.

ProductList.jsx → Create file ProductList.jsx in Components folder (in src)

```

function ProductList({products}){
  return (
    <div>

```

```

    <h2>Inventory</h2>

    {
    products.map(product=>(
        <div key={product.id}>
            <h3>{product.name}</h3>
            <p>
                Price: ₹{product.price}
            </p>
            <p>
                Quantity: {product.quantity}
            </p>
        </div>
    ))
    }

</div>

)

}

export default ProductList;

```

Step 7: Main Application (useState, useEffect, useMemo, useContext)

→ Purpose of App.jsx:-

App.jsx is responsible for:

- Storing all products.
- Storing the search text.
- Receiving the current theme from `ThemeContext`.
- Adding new products.

- Filtering products based on search.
- Calculating the total inventory value.
- Displaying all components.

App.jsx → Replace the entire code with

```
import {  
  useState,  
  useEffect,  
  useMemo,  
  useContext  
} from "react";  
  
import ProductForm from "../Components/ProductForm";  
import ProductList from "../Components/ProductList";  
import SearchBar from "../Components/SearchBar";  
import {ThemeContext} from "../Context/ThemeContext";  
  
function App(){  
  const [products,setProducts]=useState([]);  
  const [search,setSearch]=useState("");  
  const {darkMode,toggleTheme}=useContext(ThemeContext);  
  function addProduct(product){  
    setProducts(prev=>[...prev,product]);  
  }  
  
  useEffect(()=>{  
    console.log("Inventory Updated");  
  },[products]);
```

```

const filteredProducts = useMemo(()=>{
  return products.filter(product=>
    product.name
      .toLowerCase()
      .includes(search.toLowerCase())
  );
},[products,search]);

const totalValue = useMemo(()=>{
  return products.reduce((total,item)=>
    total+(item.price*item.quantity),0
  );
},[products]);

return (
  <div style={{
    background:darkMode?"black":"white",
    color:darkMode?"white":"black"
  }} >
    <h1>Inventory Management</h1>
    <button onClick={toggleTheme}>
      {darkMode?
        "Light Mode":"Dark Mode"
      }
    </button>

```

```

        <SearchBar
            setSearch={setSearch}
        />

        <ProductForm
            addProduct={addProduct}
        />

        <ProductList
            products={filteredProducts}
        />

        <h2>Total Inventory Value: ₹ {totalValue}</h2>

    </div>

)
}

export default App;

```

Step 8: Render Application

main.jsx --> Replace the code with below one.

```

import { createRoot } from "react-dom/client";

import "./index.css";

import App from "./App";

import "./App.css";

import { ThemeProvider } from "./Context/ThemeContext";

createRoot(document.getElementById("root")).render(

    <ThemeProvider>

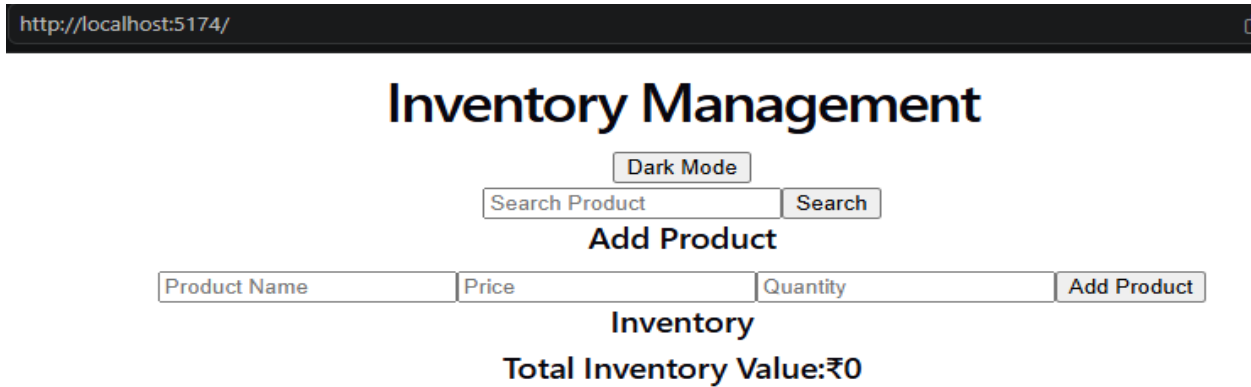
```

```
<App />
```

```
</ThemeProvider>
```

```
);
```

Initial Output:-



http://localhost:5174/

Inventory Management

Dark Mode

Search Product Search

Add Product

Product Name	Price	Quantity	Add Product
--------------	-------	----------	-------------

Inventory

Total Inventory Value: ₹0

Now Add 2 Products:-

Input 1:-

Product Name: Laptop
Price: 50000
Quantity: 3

Input 2:-

Product Name: Desktop Computers
Price: 50000
Quantity: 3

Output:-

Inventory Management

Dark Mode

Search Product

Search

Add Product

Product Name

Price

Quantity

Add Product

Inventory

Laptop

Price: ₹50000

Quantity: 3

Desktop Computers

Price: ₹50000

Quantity: 3

Total Inventory Value: ₹300000

Search Output:-

Search:

Laptop

Result:

http://localhost:5174/

Inventory Management

Dark Mode

Laptop

Search

Add Product

Product Name

Price

Quantity

Add Product

Inventory

Laptop

Price: ₹50000

Quantity: 3

Total Inventory Value: ₹300000

Theme Toggle Output:-

Light Mode:

Background: White & Text: Black

⌂ http://localhost:5174/

Inventory Management

Dark Mode

Search Product Search

Add Product

Product Name	Price	Quantity	Add Product
--------------	-------	----------	-------------

Inventory

Laptop

Price: ₹50000
Quantity: 3

Desktop Computer

Price: ₹50000
Quantity: 3

Total Inventory Value: ₹300000

Dark Mode:

Background: Black & Text: White

⌂ http://localhost:5174/

Inventory Management

Light Mode

Search Product Search

Add Product

Product Name	Price	Quantity	Add Product
--------------	-------	----------	-------------

Inventory

Laptop

Price: ₹50000
Quantity: 3

Desktop Computer

Price: ₹50000
Quantity: 3